



**Ghulam Ishaq Khan Institute
of Engineering Sciences and Technology**

Faculty of Chip DV

Design and FPGA Implementation of 73-Tap FIR Filter

Couse: CHIP-DV

Submitted by:

Abdul Samad Abbasi (25ins-01030)

Abdur Rehman Safdar (25ins-01033)

Project Supervisor: Engr. Mashood

Final Project

September 22, 2026

Abstract

This report presents the design, verification, and FPGA implementation of a 73-tap Finite Impulse Response (FIR) filter. The filter can process 1 MSps within a $1\ \mu\text{s}$, using 16-bit fixed-point coefficients Q(2,14), a Q(3,10) input format, and a Q(3,13) output format with explicit overflow handling. Rather than instantiating 73 parallel multipliers, the design exploits the filter's linear-phase symmetry ($h[k] = h[72 - k]$) to pre-add symmetric input pairs, reducing unique coefficient multiplications from 73 to 37. To do this we use two of 37×1 multiplexers which select the symmetric pair and add them. So a single adder is used repeatedly. Then sum multiplied with the coefficients and result feed to accumulator, controlled by a five-state Moore FSM, consuming only 40 of the available 100 clock cycles per sample at 100 MHz, a 60% timing margin. The design is validated against a bit-accurate C++ golden reference model and a self-checking directed testbench, then demonstrated live on FPGA Nexys A7 (Xilinx Artix-7) board, with output samples selectable via switches and displayed on LEDs. This report covers the fixed-point number system, the resource-minimization strategy, RTL architecture, resource and critical-path estimation, and hardware bring-up issues encountered during FPGA deployment.

Contents

1	Introduction	4
1.1	Project Timeline	4
2	Problem Specification and Design Constraints	5
3	Fixed-Point Number System Design	5
4	Golden Reference Model	6
5	Architectural Design Conception	7
5.1	Exploiting Coefficient Symmetry	7
5.2	Multiplier MAC	8
5.3	Block Diagram	8
5.4	Control FSM	8
6	RTL Implementation	9
7	Resource Estimation and Critical Path Analysis	11
7.1	Register count	11
7.2	Memory usage	11
7.3	LUT usage and utilization	11
8	Verification Methodology	12
9	FPGA Implementation	12
10	Difficulties Encountered and Their Solutions	13
10.1	Testbench output did not match the golden model	13
10.2	Supplying input samples on hardware	13
10.3	Data files not found during synthesis	13
10.4	Results too fast to observe on hardware	13
10.5	Two least significant bits of the displayed output were wrong	14
11	Results	14
11.1	Test signal	14
11.2	Simulation Results	14
12	Conclusion	15
A	Simulation Results	16
B	Filter Coefficients	19
C	Repository and Deliverable Map	19

List of Figures

1	FIR Filter Block Diagram	4
2	Timeline and milestones of Project	4
3	Fixed point Coefficients graph	6
4	Fixed point input samples graph	6
5	Fixed point output graph	7
6	Fixed point output graph(last 33 values)	7
7	Block Diagram of Project	8
8	Control FSM state diagram	8
9	Simulation Waveform of Output samples	12

List of Tables

1	Fixed-point formats used in the datapath	5
2	MAC schedule (representative cycles)	9
3	RTL module summary	9
4	Structural flip-flop count of the filter core	11
5	Memory implementation as reported by synthesis	11
6	Device utilization, xc7a100tcsg324-1	11
7	Synthesised cell count by instance	12
8	Simulation verification summary	14
9	Representative sample-by-sample comparison (DUT vs. golden model)	15
10	Full sample-by-sample comparison (DUT vs. golden model)	16
11	Unique filter coefficients $h[0..36]$	19

1 Introduction

An FIR filter computes each output sample as a weighted sum of the current and past $N - 1$ input samples:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n - k] \quad (1)$$

where $h[k]$ are the filter coefficients (the impulse response) and N is the filter length. Unlike IIR filters, an FIR filter has no feedback path, which guarantees bounded-input bounded-output (BIBO) stability, and when the coefficients are symmetric ($h[k] = h[N - 1 - k]$) the filter has an exactly linear phase response, a property widely required in applications where waveform shape must be preserved (audio, communications, biomedical signal processing).

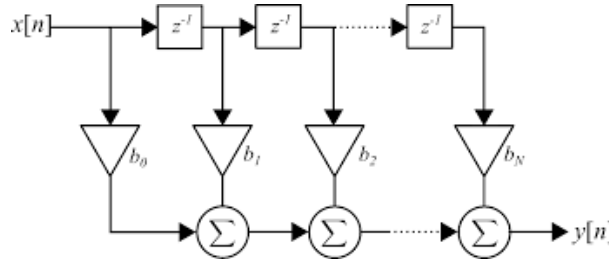


Figure 1: FIR Filter Block Diagram

1. **Coefficient symmetry exploitation.** Because the given coefficient set is symmetric ($h[k] = h[72 - k]$, with an unpaired centre tap $h[36]$), the two samples that share a coefficient are added *before* the multiplication, halving the number of unique multiplications from 73 to 37.
2. **Time multiplexing.** The 1 μ s budget spans 100 clock cycles at a 100 MHz system clock, so the 37 multiply-accumulate operations do not need to happen in parallel. A single multiplier and a single accumulator, reused sequentially under the control of a finite state machine, reduce the DSP-slice requirement from a theoretical 73 (or 37) down to exactly **one**.

1.1 Project Timeline

The project was completed over a 9-day development cycle, structured as follows:

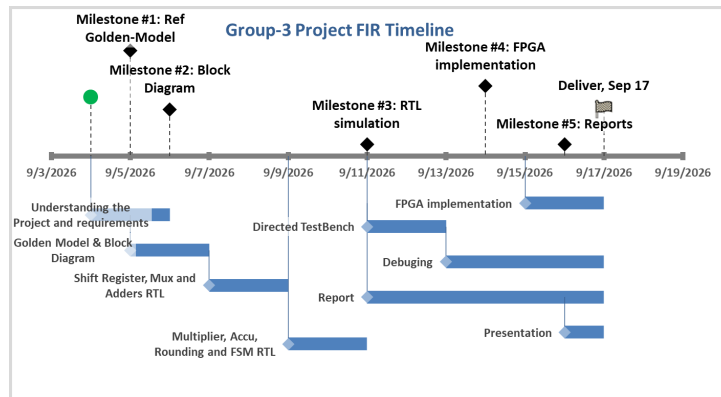


Figure 2: Timeline and milestones of Project

2 Problem Specification and Design Constraints

The project problem statement specifies the following design constraints [1]:

- **Sample rate:** Samples arrive at 1 MSps, the complete filtering operation for one incoming sample must finish within 1 μ s.
- **Coefficient length:** Filter coefficients must be converted to a fixed-point representation using no more than 16 bits.
- **Input format:** The input sample $x[n]$ is a signed fixed-point number with 2 integer bits and 10 fraction bits.
- **Output format:** The output $y[n]$ must be produced in $Q(3, 13)$ format, with the possibility of arithmetic overflow explicitly handled (i.e. saturated, not wrapped).
- **Coefficients:** A fixed set of 73 floating-point coefficients is provided. Inspection of the given values confirms the filter is symmetric, $h[k] = h[72 - k]$, with the centre tap $h[36] = 1.0$ exactly – the signature of a linear-phase, unity-DC-adjacent design.

The required deliverables are:

1. A C/C++ golden reference model with 100 generated test samples.
2. Study report.
3. Synthesizable Verilog HDL for the filter.
4. The test bench for the FIR filter.
5. FPGA demonstration on Nexys A7 FPGA Board.

3 Fixed-Point Number System Design

All three datapath quantities – input, coefficient, and output – are represented in 16-bit signed $Q(m, n)$ fixed-point format, where by convention m *includes* the sign bit. Table 1 summarizes the chosen formats.

Table 1: Fixed-point formats used in the datapath

Signal	Format	Breakdown	Range
Input $x[n]$	$Q(3, 10)$	1s+2i+10f	$[-4.0, +3.999]$
Coefficient $h[k]$	$Q(2, 14)$	1s+1i+14f	$[-2.0, +1.999]$
Output $y[n]$	$Q(3, 13)$	1s+2i+13f	$[-4.0, +3.999]$

The coefficient format $Q(2, 14)$ was chosen (rather than, e.g., $Q(3, 13)$) because every coefficient in the given set lies in $[-1, 1]$ (the largest magnitude value is exactly 1.0, the centre tap), so only a single integer bit is enough to represent the coefficients. The freed bits are allocated to fraction, maximizing coefficient precision within the 16-bit budget mandated by the problem statement. Quantization to fixed point uses

$$\text{fixed_value} = \text{round}(\text{real_value} \times 2^n), \quad (2)$$

where n is the number of fraction bits for that signal.

Product and accumulator width. Multiplying a $Q(3, 10)$ pre-summed pair by a $Q(2, 14)$ coefficient produces a 32-bit product with $10 + 14 = 24$ fraction bits. Accumulating 37 such 32-bit signed products requires $\lceil \log_2 37 \rceil \approx 5.2 \rightarrow 6$ extra guard bits to guarantee no overflow *during* accumulation, giving a $32 + 6 = 38$ -bit accumulator. The final 38-bit, 24-fraction-bit accumulator value is converted to the 16-bit, 13-fraction-bit output by (i) adding 2^{10} (half the LSB about to be discarded) for round-to-nearest, (ii) arithmetically right-shifting by $24 - 13 = 11$ bits, and (iii) saturating the result to the 16-bit signed range $[-32768, 32767]$ instead of permitting silent wraparound – directly satisfying the overflow-handling requirement of the problem statement.

4 Golden Reference Model

Per the first required deliverable, a bit-accurate golden reference model was implemented in both C and C++ (functionally identical) *before* any RTL was written, and is treated as the single source of truth for every subsequent verification stage. The model:

1. Quantizes all 73 floating-point coefficients to $Q(2, 14)$ using Eq. (2) and writes them, one per line, as 16-bit hexadecimal values to `coeffs_fixed.txt` – the exact format consumed by the Verilog coefficient ROM via `$readmemh`.

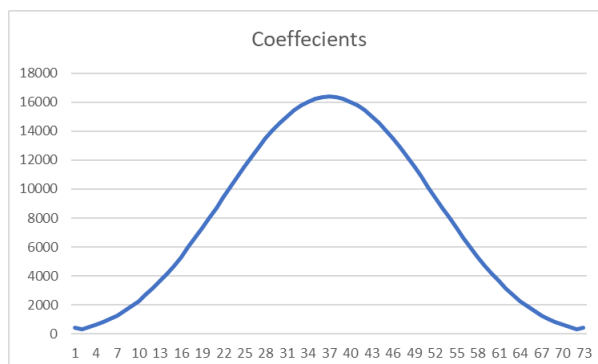


Figure 3: Fixed point Coefficients graph

2. Generates 100 test input samples, a sinusoid of period 25 samples at near-full-scale amplitude – quantizes them to $Q(3, 10)$, and writes them to `input_samples.txt` in the same hexadecimal, one-per-line format.

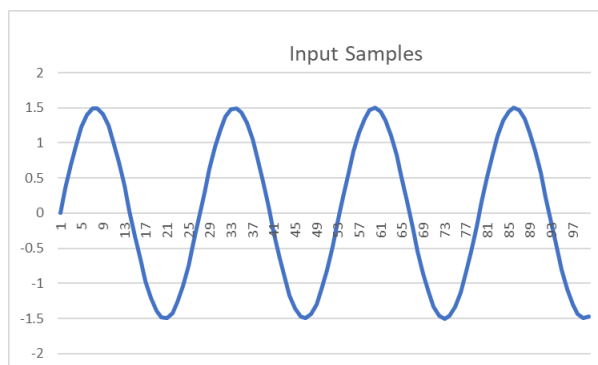


Figure 4: Fixed point input samples graph

3. Computes the reference output by simulating the filter in software: a tapped delay line (shift register model) combined with a wide accumulator, which by construction cannot

overflow for this problem size and thus provides an exact, arbitrary-precision-equivalent accumulation against which the hardware’s 38-bit accumulator can later be checked.

4. Rescales every accumulated result using the identical round/shift/saturate procedure, and writes the resulting 100 expected outputs, again as 16-bit hexadecimal, to `golden_output.txt`.

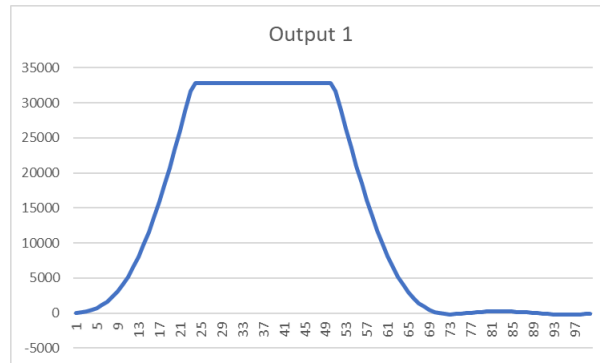


Figure 5: Fixed point output graph

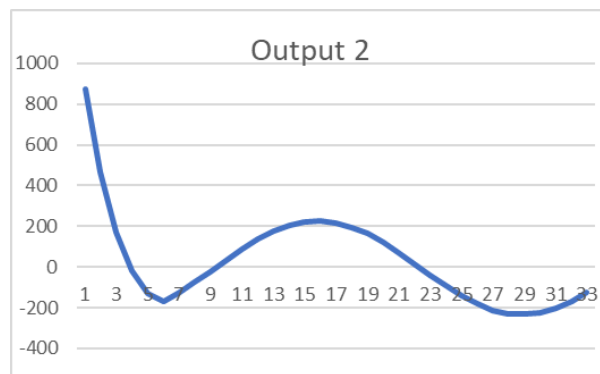


Figure 6: Fixed point output graph(last 33 values)

5 Architectural Design Conception

5.1 Exploiting Coefficient Symmetry

Because $h[k] = h[72 - k]$, the direct-form sum

$$y[n] = \sum_{k=0}^{72} h[k] x[n - k] \quad (3)$$

can be re-grouped as

$$y[n] = \sum_{k=0}^{35} h[k] \left(x[n - k] + x[n - 72 + k] \right) + h[36] x[n - 36], \quad (4)$$

36 symmetric pairs of taps that share a coefficient, plus one unpaired centre tap. Pre-adding each pair before the single shared multiplication reduces the number of *unique* multiply operations from 73 to $36 + 1 = 37$ – an almost 50% reduction.

5.2 Multiplier MAC

The $1\text{ }\mu\text{s}$ per-sample budget at a 100 MHz system clock corresponds to 100 clock cycles – far more than the 37 sequential multiply-accumulate operations required by Eq. (4). The architecture therefore instantiates exactly one multiplier and one accumulator, reused for all 37 MAC operations under the control of a dedicated finite state machine, rather than 37 (or 73) parallel multiply-accumulate units. This is the single largest resource-saving decision in the design and is what allows the entire filter core to require only a single DSP48 hardware multiplier (Section 7).

5.3 Block Diagram

A 73-stage shift register holds the most recent 73 input samples. On each of the 37 MAC cycles, a 37:1 tap-pair-select multiplexer presents the pair $(x[n - k], x[n - 72 + k])$ addressed by a 6-bit cycle counter; the pair is summed, multiplied by the coefficient ROM's entry at the same address, and accumulated. Once all 37 cycles complete, the accumulator's final value is rounded, shifted, and saturated into the 16-bit output register.

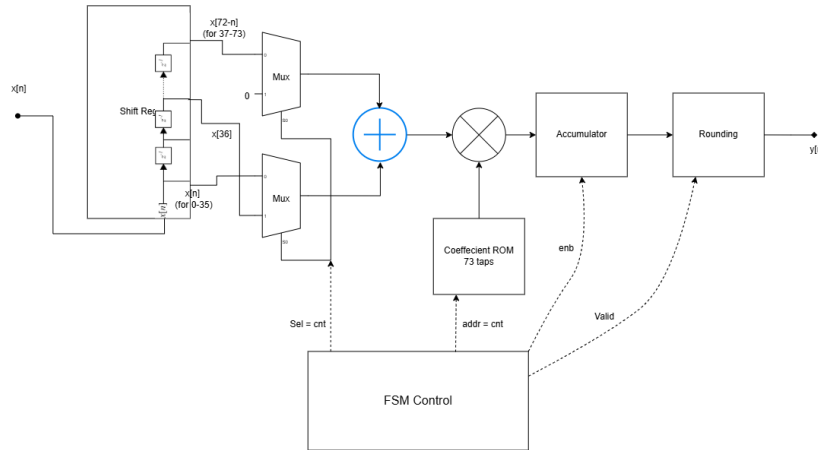


Figure 7: Block Diagram of Project

5.4 Control FSM

Following figure shows the five-state Moore FSM that sequences the datapath. IDLE waits for an externally supplied `sample_valid` pulse. LOAD shifts the new sample into the delay line and clears the accumulator. COMPUTE runs for exactly 37 cycles ($\text{cycle_cnt} = 0 \dots 36$), with the accumulator enabled every cycle. ROUND_SAT is a single-cycle pause that lets the accumulator's final value settle.

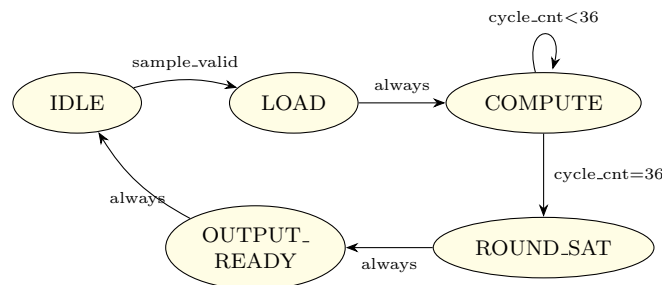


Figure 8: Control FSM state diagram

OUTPUT_READY asserts the load pulse that latches the rounded result into the output register, then returns to IDLE.

At each COMPUTE cycle $k = 0, \dots, 36$, the tap-pair multiplexer selects `tap_low = taps[k]` and `tap_high = taps[72 - k]` (with `tap_high` forced to zero at $k = 36$, the unpaired centre tap), and the coefficient ROM is addressed by the same k , guaranteeing the pair and its shared coefficient always arrive together. Table 2 illustrates the schedule.

Table 2: MAC schedule (representative cycles)

Cycle k	<code>tap_low</code>	<code>tap_high</code>	Coeff. addr.
0	<code>taps[0]</code>	<code>taps[72]</code>	0
1	<code>taps[1]</code>	<code>taps[71]</code>	1
2	<code>taps[2]</code>	<code>taps[70]</code>	2
$\vdots$	$\vdots$	$\vdots$	$\vdots$
35	<code>taps[35]</code>	<code>taps[37]</code>	35
36	<code>taps[36]</code>	0 (unpaired)	36

At the system clock of 100 MHz (period 10 ns), the full 40-cycle schedule consumes 400 ns of the 1 μ s budget – a **60% timing margin**.

6 RTL Implementation

The design is partitioned into eight independent, individually simulatable submodules plus a purely-structural top-level wiring module, summarized in Table 3.

Table 3: RTL module summary

Module	Type	Function
<code>shift_reg</code>	Sequential	73-stage, 16-bit-wide serial-in/parallel-out shift register holding $x[n] \dots x[n - 72]$.
<code>tap_pair_select</code>	Combinational	37:1 multiplexer pair selecting <code>taps[k]</code> and <code>taps[72 - k]</code> addressed by <code>cycle_cnt</code> .
<code>tap_adder</code>	Combinational	Adds the selected symmetric pair. No width growth is required: the $Q(3, 10)$ container already carries 3 guard bits above the 13 meaningful bits.
<code>coeff_rom</code>	ROM	37×16 -bit distributed ROM holding $h[0..36]$, loaded at elaboration time from <code>coeffs_fixed.txt</code> via <code>\$readmemh</code> , addressed by <code>cycle_cnt</code> .
<code>tap_multiplier</code>	Combinational	$16 \times 16 \rightarrow 32$ -bit signed multiply, mapping it directly onto a hard DSP48 macro.
<code>accumulator</code>	Sequential	38-bit signed accumulator, cleared during LOAD, enabled for all 37 COMPUTE cycles, holds otherwise.
<code>round_saturate</code>	Mixed	Combinational round/shift/saturate logic feeding a registered output, also generates the <code>valid</code> status flag as a one-cycle-delayed echo of the load pulse, exactly aligned with the cycle the output register actually captures the new value.
<code>control_fsm</code>	Sequential + Combinational	Five-state Moore FSM (Fig. 8) generating every datapath enable/select signal and the 6-bit cycle counter.
<code>fir_top</code>	Structural	Wiring, instantiates and connects all of the above, contains no independent datapath logic of its own.

Listing 1 shows the finalized control FSM implementation, which is the central coordinating block of the design.

```
1 always@(*) begin
2     shift_en    = 1'b0;
3     clear       = 1'b0;
4     acc_enb     = 1'b0;
5     out_enb     = 1'b0;
6     next_state = state;
7     case (state)
8         IDLE:
9
10            if (sample_valid) begin
11                next_state = LOAD;
12            end
13        LOAD: begin
14            shift_en    = 1'b1;
15            clear       = 1'b1;
16            next_state = COMPUTE;
17        end
18
19        COMPUTE: begin
20            acc_enb = 1'b1;
21            if (cycle_cnt == 6'd36) begin
22                next_state = ROUNDING;
23            end
24        end
25
26        ROUNDING:
27            next_state = OUTPUT_READY;
28
29        OUTPUT_READY: begin
30            out_enb     = 1'b1;
31            next_state = IDLE;
32        end
33        default:
34            next_state = IDLE;
35    endcase
36 end
```

Listing 1: Control FSM: next-state and output logic

7 Resource Estimation and Critical Path Analysis

7.1 Register count

Table 4: Structural flip-flop count of the filter core

Register	Bits
Shift register (<code>shift_out[0:72]</code> , $73 \times 16\text{b}$)	1168
Accumulator (38b)	38
Output register <code>y_out</code> (16b) + <code>valid</code> (1b)	17
FSM state (3b) + cycle counter (6b)	9
Core filter total	1232

Synthesis yields **1219** **FDRE** primitives, of which $73 \times 16 = 1168$ – **96 %** – are the shift register, all remaining registers, including the board-level debounce counter and address pointers, account for 51.

7.2 Memory usage

Table 5: Memory implementation as reported by synthesis

Memory	Size	Implemented as
Coefficient ROM (<code>coeff_rom</code>)	64×16	LUT (distributed ROM)
Input-sample ROM (<code>input_rom</code>)	128×16	Block RAM (1 RAMB18E1)
Output memory (<code>out_ram</code>)	128×16	RAM64M $\times 10$, RAM64X1D $\times 2$

The output memory infers as distributed RAM because it is written one address per cycle, including during its post-reset clear; a single-cycle clear of all 100 words would have no RAM primitive to map onto and would cost roughly 1600 extra flip-flops.

7.3 LUT usage and utilization

Table 6: Device utilization, `xc7a100tcsg324-1`

Resource	Used	Available	Utilization
Slice LUTs	553	63,400	0.87 %
as logic	509	63,400	0.80 %
as distributed RAM	44	19,000	0.23 %
Slice registers (FDRE)	1219	126,800	0.96 %
F7 multiplexers	129	31,700	0.41 %
DSP48E1	1	240	0.42 %
Block RAM (RAMB18)	1	270	0.37 %

Table 7: Synthesised cell count by instance

Instance	Module	Cells
top (whole design)	—	1962
fir_shift	shift_reg	1601
fir_fsm	control_fsm	118
fir_rounding	rounding	102
fir_adder	tap_adder	4
fir_acc	accumulator	2

8 Verification Methodology

Verification follows the golden-model-referenced methodology:

1. **Directed RTL testbench:** loads `input_samples.txt` and `golden_output.txt` via `$readmemh`, feeds the design under test, one sample at a time by pulsing `sample_valid` for exactly one cycle, waits for `output_valid`, and compares `y_out` against the corresponding golden value, reporting a per-sample PASS/FAIL and a final summary (total passed / total samples). The console output is shown in Appendix.
2. **Waveform:** The most common reason in a design of mismatches are shift-register timing (an off-by-one-cycle load), incorrect rounding-bit placement, and sign-extension errors at the multiplier/accumulator boundary.

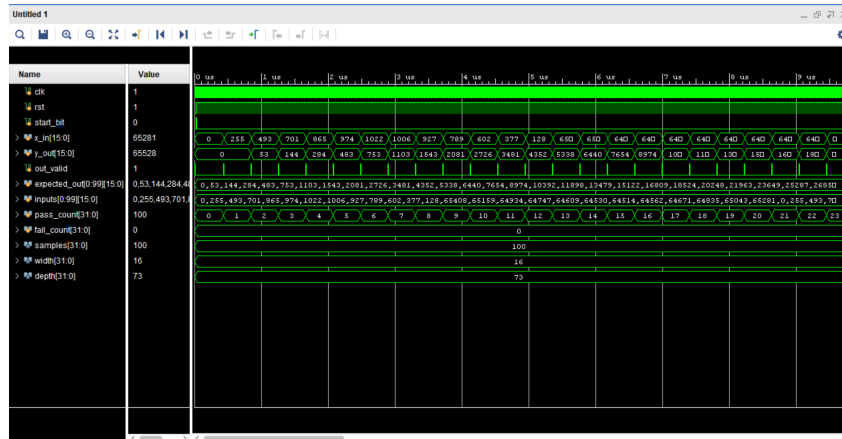


Figure 9: Simulation Waveform of Output samples

9 FPGA Implementation

The design was demonstrated on a Nexys A7 board (Xilinx Artix-7, `xc7a100tcsg324-1`) [6], using its 100 MHz onboard oscillator as the system clock. The 100 golden-model input samples are stored in an inferred ROM; a debounced push-button (`start_bit`) triggers streaming of the sample sequence through the filter core, one sample every `output_valid` pulse. Filtered outputs are written into a simple output memory as they are produced, the output memory's read-address port is driven by the board's slide switches and its data output drives the LEDs, exactly as suggested in the problem statement's validation

guidance, allowing any of the 100 filtered samples to be inspected on demand by setting the corresponding switch value.

10 Difficulties Encountered and Their Solutions

There are some problems that cost significant debugging time during the project.

10.1 Testbench output did not match the golden model

Cause. The port declarations of the multiplier and accumulator used plain `logic` rather than `logic signed`. A signed wire attached to an unsigned port becomes unsigned inside the module. The multiplication was therefore performed as an unsigned operation, and a negative operand was interpreted as a large positive number. This is why the mismatch happened.

Solution. Every datapath port carrying a two's-complement value was declared `signed`:

```
// before:  input logic      [15:0] sum, coeff;  
// after :  input logic signed [15:0] sum, coeff;
```

After this change all 100 samples matched the golden reference exactly.

10.2 Supplying input samples on hardware

Cause. A testbench drove the input samples but it is not synthesizable. It exists only inside the simulator, so on the FPGA there is nothing driving the input port.

Solution. The sample source was moved inside the design. A read-only memory was added to the top module from `input_samples.txt` via `$readmemh`, and addressed by a read pointer that increment by one each time an output sample completes.

10.3 Data files not found during synthesis

Cause. The coefficient and input-sample text files were read correctly in simulation, but synthesis could not find them.

Solution. The data files were made available in the synthesis run directory as well, so that the memory contents are initialised identically in both flows.

10.4 Results too fast to observe on hardware

Cause. The design is simply far faster than the output device. The complete 100-sample run finishes in roughly 41 μ s. Then how we can observe the output on LEDs?

Solution. An output memory was added, and each filtered sample is written into it as it is produced. Once the run is complete the stored results can be shown on LEDs. So setting the switches to any value 0...99 displays the corresponding output sample.

10.5 Two least significant bits of the displayed output were wrong

Symptom.

Cause. Roughly half of the stored samples displayed an incorrect value on the LEDs, always short by exactly 2, while the rest were correct. In the XDC file the second LED, `led[1]`, had been assigned a wrong package pin.

Solution. The pin assignment was corrected against the board’s reference manual. A constant offset of a power of two, appearing on some samples but not others, points to a single mis-mapped output bit.

11 Results

11.1 Test signal

The primary test signal used for both simulation and the FPGA demonstration is the 100-sample sequence generated by the golden model: a near-full-scale sinusoid of period 25 samples, quantized to $Q(3, 10)$. This signal was chosen because its periodicity makes shift-register timing and load-alignment bugs immediately visible (a correct filter produces an equally periodic output), while its near-full-scale amplitude also exercises the design’s rounding behaviour without triggering saturation, providing a clean baseline regression set distinct from the saturation-focused stress vector described next.

11.2 Simulation Results

The design was verified against all 100 test vectors generated by the C++ golden reference model, covering both the linear (non-saturating) response region and the saturation region where the accumulated output exceeds the 16-bit signed range and is clamped to ± 32767 . The self-checking testbench compared every computed `y_out` sample against the corresponding golden output on a per-sample basis.

Table 8: Simulation verification summary

Metric	Value
Total test vectors	100
Samples passed	100
Samples failed	0
Pass rate	100%

Table 9 shows a representative subset of the results, spanning the rising edge of the response, the saturated plateau (samples 31–41, where `y_out` is clamped at 32767), and the settling/negative-going tail.

Table 9: Representative sample-by-sample comparison (DUT vs. golden model)

Sample	DUT y_out	Expected	Result
0	0	0	PASS
10	3481	3481	PASS
20	16809	16809	PASS
30	32121	32121	PASS
31	32767	32767	PASS (saturated)
41	32767	32767	PASS (saturated)
42	32181	32181	PASS
60	5323	5323	PASS
80	−60	−60	PASS
99	−8	−8	PASS

All 100 samples matched the golden reference exactly, with zero mismatches, confirming bit-accurate functional correctness of the RTL implementation, including correct handling of the saturation/rounding logic during the high-amplitude portion of the response (samples 31–41).

12 Conclusion

This report has documented the complete design flow for a 73-tap symmetric FIR filter meeting a 1 MSa s^{-1} real-time constraint under a minimum-FPGA-resource objective: from fixed-point number system selection, through a golden C/C++ reference model, an architecture that combines coefficient-symmetry exploitation with time multiplexing to reduce DSP-slice usage from a theoretical 73 down to exactly one, RTL implementation and resource/critical-path analysis, a pipelining discussion, a golden-model-referenced verification methodology, and a working FPGA demonstration – including an honest account of the hardware bring-up issues (reset polarity, debounce one-shot behaviour, and cross-run shift-register state) that had to be resolved between a simulation-clean design and a correctly functioning physical system.

References

- [1] Chip Design and Verification Course Staff, “Complex Engineering Problem – Group A: 73-Tap FIR Filter,” Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI), problem statement handout.
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.
- [3] Xilinx, Inc., *7 Series DSP48E1 Slice User Guide*, UG479.
- [4] Xilinx, Inc., *7 Series FPGAs Configurable Logic Block User Guide*, UG474.
- [5] Xilinx, Inc., *Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics*, DS181.
- [6] Digilent, Inc., *Nexys A7 FPGA Board Reference Manual*.
- [7] r/FPGA community discussion, “FIR filter implementation,” *Reddit*. [Online]. Available: https://www.reddit.com/r/FPGA/comments/a4arcg/fir_filter_implementation/
- [8] R. Barak, “FIR-Filter,” *GitHub repository*. [Online]. Available: <https://github.com/rombarak/FIR-Filter>
- [9] A. He, “Implementing a FIR Filter in Verilog: A Beginner’s Guide to FPGA Digital Signal Processing,” *Medium*. [Online]. Available: <https://medium.com/@ahe24mobile/implementing-a-fir-filter-in-verilog-a-beginners-guide-to-fpga-digital-signal-processing-9ee05ad0da41>
- [10] VHDLwhiz, “Part 2: Finite Impulse Response (FIR) Filters.” [Online]. Available: <https://vhdlwhiz.com/part-2-finite-impulse-response-fir-filters/>
- [11] Anthropic, *Claude* (AI assistant). Used for RTL debugging and optimization. [Online]. Available: <https://claude.ai>

A Simulation Results

Table 10: Full sample-by-sample comparison (DUT vs. golden model)

Sample	DUT y _{out}	Expected	Result
0	0	0	PASS
1	53	53	PASS
2	144	144	PASS
3	284	284	PASS
4	483	483	PASS
5	753	753	PASS
6	1103	1103	PASS
7	1543	1543	PASS
8	2081	2081	PASS
9	2726	2726	PASS
10	3481	3481	PASS
11	4352	4352	PASS

Continued on next page

Table 10 – continued from previous page

Sample	DUT	y_out	Expected	Result
12		5338	5338	PASS
13		6440	6440	PASS
14		7654	7654	PASS
15		8974	8974	PASS
16		10392	10392	PASS
17		11898	11898	PASS
18		13479	13479	PASS
19		15122	15122	PASS
20		16809	16809	PASS
21		18524	18524	PASS
22		20248	20248	PASS
23		21963	21963	PASS
24		23649	23649	PASS
25		25287	25287	PASS
26		26859	26859	PASS
27		28345	28345	PASS
28		29728	29728	PASS
29		30992	30992	PASS
30		32121	32121	PASS
31		32767	32767	PASS (saturated)
32		32767	32767	PASS (saturated)
33		32767	32767	PASS (saturated)
34		32767	32767	PASS (saturated)
35		32767	32767	PASS (saturated)
36		32767	32767	PASS (saturated)
37		32767	32767	PASS (saturated)
38		32767	32767	PASS (saturated)
39		32767	32767	PASS (saturated)
40		32767	32767	PASS (saturated)
41		32767	32767	PASS (saturated)
42		32181	32181	PASS
43		31051	31051	PASS
44		29783	29783	PASS
45		28391	28391	PASS
46		26894	26894	PASS
47		25310	25310	PASS
48		23657	23657	PASS
49		21955	21955	PASS
50		20226	20226	PASS
51		18488	18488	PASS
52		16762	16762	PASS
53		15067	15067	PASS
54		13420	13420	PASS
55		11838	11838	PASS
56		10335	10335	PASS

Continued on next page

Table 10 – continued from previous page

Sample	DUT y_out	Expected	Result
57	8923	8923	PASS
58	7613	7613	PASS
59	6411	6411	PASS
60	5323	5323	PASS
61	4352	4352	PASS
62	3496	3496	PASS
63	2755	2755	PASS
64	2122	2122	PASS
65	1594	1594	PASS
66	1160	1160	PASS
67	813	813	PASS
68	542	542	PASS
69	338	338	PASS
70	190	190	PASS
71	88	88	PASS
72	22	22	PASS
73	8	8	PASS
74	−8	−8	PASS
75	−22	−22	PASS
76	−35	−35	PASS
77	−46	−46	PASS
78	−55	−55	PASS
79	−59	−59	PASS
80	−60	−60	PASS
81	−57	−57	PASS
82	−51	−51	PASS
83	−41	−41	PASS
84	−29	−29	PASS
85	−15	−15	PASS
86	0	0	PASS
87	15	15	PASS
88	29	29	PASS
89	41	41	PASS
90	51	51	PASS
91	57	57	PASS
92	60	60	PASS
93	59	59	PASS
94	55	55	PASS
95	46	46	PASS
96	35	35	PASS
97	22	22	PASS
98	8	8	PASS
99	−8	−8	PASS

B Filter Coefficients

Because the filter is symmetric ($h[k] = h[72 - k]$), only the 37 unique values $h[0..36]$ are listed; $h[37..72]$ are the mirror image. Table 11 gives both the original floating-point value and its quantized $Q(2, 14)$ fixed-point (16-bit signed hexadecimal) equivalent, as written to `coeffs.fixed.txt`.

Table 11: Unique filter coefficients $h[0..36]$

k	$h[k]$ (float)	$Q(2, 14)$ hex
0	0.025804122	0x01a7
1	0.020552684	0x0151
2	0.028396631	0x01d1
3	0.037923047	0x026d
4	0.049296639	0x0328
5	0.062671826	0x0403
6	0.078188509	0x0501
7	0.095967794	0x0624
8	0.116107786	0x076e
9	0.138679546	0x08e0
10	0.163723333	0x0a7a
11	0.191245225	0x0c3d
12	0.221214225	0x0e28
13	0.253559961	0x103a
14	0.288171034	0x1271
15	0.324894126	0x14cb
16	0.363533898	0x1744
17	0.403853725	0x19d9
18	0.445577304	0x1c84
19	0.488391123	0x1f42
20	0.531947770	0x220b
21	0.575870065	0x24db
22	0.619755930	0x27aa
23	0.663183933	0x2a72
24	0.705719403	0x2d2b
25	0.746921012	0x2fce
26	0.786347677	0x3254
27	0.823565670	0x34b5
28	0.858155769	0x36ec
29	0.889720317	0x38f1
30	0.917890027	0x3abf
31	0.942330394	0x3c4f
32	0.962747575	0x3d9e
33	0.978893597	0x3ea6
34	0.990570797	0x3f66
35	0.997635377	0x3fd9
36	1.000000000	0x4000

C Repository and Deliverable Map

All source files referenced in this report – the golden model (`golden_model.c/.cpp`), the nine RTL modules of Table 3, the self-checking testbench, and the board-level top-level wrapper – are maintained under git version control with incremental, commented commits, per the submission requirement of the problem statement.